

Smart Expense Approval Platform

Problem Statement

1.1 Slow Expense Submissions

Employees sometimes experience delays because the application and database take longer to process requests under increasing workload.

Reasoning:

- Increasing number of requests
- Application and database processing load
- Higher latency

1.2 Missed Manager Updates

Managers may miss important expense updates because the current notification process is not sufficiently reliable or real-time.

Reasoning:

- Notification delays or failures
- No reliable asynchronous event processing
- Important updates may be missed

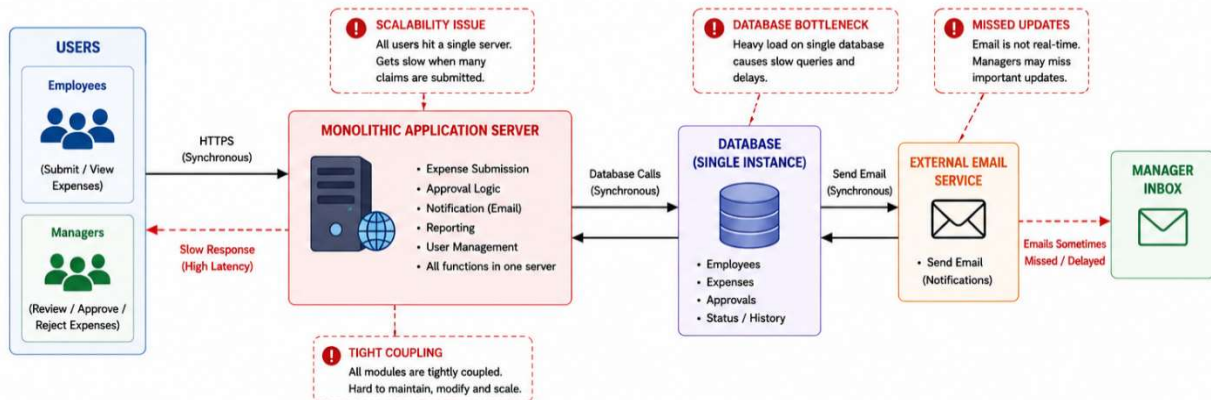
1.3 Poor Responsiveness Under High Load

The platform becomes slow or unresponsive when many employees submit expense claims simultaneously, overloading the application and database.

Reasoning:

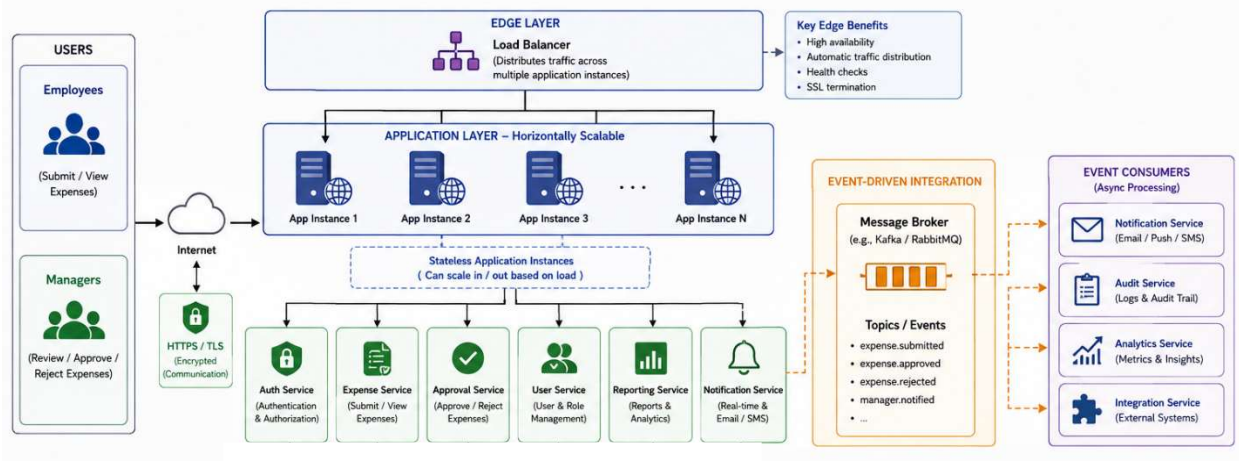
- Many simultaneous requests
- Application/database bottlenecks
- Limited scalability
- Increased latency

SMART EXPENSE APPROVAL PLATFORM – CURRENT ARCHITECTURE



Proposed Solution

1. **HTTPS** - Secures communication between employees, managers, and the platform.
2. **Load Balancer** - Distributes incoming requests across multiple application instances to prevent overload.
3. **Multiple Expense Service Instances** - Allows multiple instances to process requests simultaneously and improves scalability.
4. **Docker** - Packages the Expense Service into containers for consistent and easy deployment.
5. **Kubernetes** - Manages, scales, and automatically restarts containers when required.
6. **Database** - Stores user, expense, and approval information reliably.
7. **Message Broker** - Enables reliable asynchronous communication between services.
8. **Event-Driven Architecture** - Uses events such as ExpenseSubmitted, ExpenseApproved, and ExpenseRejected to trigger independent services.
9. **Notification Service** - Sends expense updates to managers and can use WebSocket for real-time notifications.



3. Assumptions

1. **Users** - The platform has two main user types: employees who submit expenses and managers who approve or reject them.
2. **Growing Usage** - The number of employees and expense claims will increase over time, requiring the system to scale.
3. **Secure Communication** - All user-to-platform communication is assumed to use **HTTPS**.
4. **Event-Based Updates** - Important actions such as ExpenseSubmitted, ExpenseApproved, and ExpenseRejected generate events.
5. **Message Broker** - A message broker such as Kafka, RabbitMQ, or Azure Service Bus is available for asynchronous event processing.
6. **Containerized Services** - Application services can be packaged and deployed using **Docker** containers.
7. **Scalable Infrastructure** - Multiple application instances can be deployed behind a load balancer to handle increasing traffic.
8. **Real-Time Notifications** - Managers are expected to receive important expense updates quickly; **WebSocket** may be used where real-time updates are required.
9. **High Availability** - Critical infrastructure such as the application and database is assumed to have appropriate redundancy and backup mechanisms in a production environment.